

Alexander Toles
Assignment 1 AI Disclosure
Sept. 27, 2026

To create TolesA1.cpp, I consulted ChatGPT model 5.6 on Sept. 25 and Sept. 27. The original prompts and responses are documented below. I have tested this program and can confirm it works as intended.

Sept. 25:

Prompt:

Finish this C++ code without using built-in regex, string parsing or string-to-number functions or libraries. Exactly one valid address may be extracted per line.

```
#include <iostream>
using namespace std;
bool extractIPv4(const std::string& str, unsigned long& outAddress, int& outPort){
    //on success, outAddress holds 32-bit address. outPort holds port number or -1 if not
    found
    //on failure, outAddress set to 0, outPort set to -1.
    //only digits (1-9), periods (.) and colons (:) allowed
    //candidates are formatted as four octets separated by periods. (octet.octet.octet.octet)
    with each octet being 3 digits 1-255
    //with no leading zeros unless the number is literally zero
    //an optional :port may follow the fourth octet, with the port being 1-5 digits with values
    ranging from 1-65535 with same leading zero rule
    //if colon is present, port must be correct or else entire candidate is rejected
    //candidate must match grammar in full, no partial matches or truncated matches

    return 0;
}

int main(){
    string inputText;
    unsigned long outAddress; //32-bit IPv4 address
    int outPort; //port number
    cout << "Enter noisy string of text: ";
    cin >> inputText;
    bool found = extractIPv4(inputText,outAddress,outPort);
    while(!found){
        if(found){
```

```

        cout << "IPv4 Found: " << outAddress;
        if(outPort > 0){
            cout << outPort << "\n";
        }
    }
    else{
        cout << "No address found.\n";
    }
}

//print extracted IPv4 in this format: Extracted IPv4 address: A.B.C.D (decimal value: N,
port: P) where N is the 32-bit decimal value and P is the port number or the literal text none.
return 0;
}

```

Chat GPT Response:

Here is a completed version that avoids regex, string parsing/tokenization, and string-to-number functions. It scans the input character-by-character and performs numeric conversion manually.

I also fixed the syntax error in main, changed input to getline so a whole line is processed, and made the loop terminate correctly after one extraction.

```

#include <iostream>
#include <string>
using namespace std;

```

```

bool isDigit(char c) {
    return c >= '0' && c <= '9';
}

```

```

bool isAddressChar(char c) {
    return isDigit(c) || c == '.' || c == ':';
}

```

```

// Parse one decimal number starting at pos.
// maxDigits limits the number of digits.
// Returns the position immediately after the number.
bool parseNumber(const string& str, size_t& pos, int maxDigits,
    int maxValue, int& value) {
    size_t start = pos;
    int digits = 0;
    int number = 0;

```

```
if (pos >= str.size() || !isDigit(str[pos]))  
    return false;
```

```
// Leading zero is allowed only when the number is exactly zero.  
if (str[pos] == '0') {  
    ++pos;
```

```
// A second digit means leading zero.  
if (pos < str.size() && isDigit(str[pos]))  
    return false;
```

```
    value = 0;  
    return true;  
}
```

```
while (pos < str.size() &&  
        isDigit(str[pos]) &&  
        digits < maxDigits) {  
    number = number * 10 + (str[pos] - '0');  
    ++pos;  
    ++digits;
```

```
    if (number > maxValue)  
        return false;  
}
```

```
// More digits than allowed.  
if (pos < str.size() && isDigit(str[pos]))  
    return false;
```

```
if (number < 1 || number > maxValue)  
    return false;
```

```
value = number;  
(void)start; // start is not otherwise needed  
return true;  
}
```

```
bool extractIPv4(const std::string& str,  
                 unsigned long& outAddress,  
                 int& outPort) {  
    // Failure defaults.  
    outAddress = 0;  
    outPort = -1;
```

```
for (size_t start = 0; start < str.size(); ++start) {  
    if (!isDigit(str[start]))  
        continue;
```

```
    // A candidate cannot begin in the middle of another  
    // address-like sequence.  
    if (start > 0 && isAddressChar(str[start - 1]))  
        continue;
```

```
    size_t pos = start;  
    int octet[4];
```

```
    // Parse four octets.  
    bool valid = true;
```

```
    for (int i = 0; i < 4; ++i) {  
        if (!parseNumber(str, pos, 3, 255, octet[i])) {  
            valid = false;  
            break;  
        }  
    }
```

```
    if (i < 3) {  
        if (pos >= str.size() || str[pos] != ':') {  
            valid = false;  
            break;  
        }  
        ++pos;  
    }  
}
```

```
if (!valid)  
    continue;
```

```
// Optional port.  
int port = -1;
```

```
if (pos < str.size() && str[pos] == ':') {  
    ++pos;
```

```
    // A colon requires a valid port.  
    if (!parseNumber(str, pos, 5, 65535, port)) {  
        continue;  
    }  
}
```

```
}
```

```
// The candidate must end here. This prevents partial matches  
// such as extracting 192.168.1.1 from 192.168.1.1.999.  
if (pos < str.size() && isAddressChar(str[pos]))  
    continue;
```

```
// Build the 32-bit IPv4 value manually.  
unsigned long address = 0;  
address = address * 256 + octet[0];  
address = address * 256 + octet[1];  
address = address * 256 + octet[2];  
address = address * 256 + octet[3];
```

```
outAddress = address;  
outPort = port;  
return true;  
}
```

```
return false;  
}
```

```
void printIPv4(unsigned long address, int port) {  
    unsigned long a = (address >> 24) & 255;  
    unsigned long b = (address >> 16) & 255;  
    unsigned long c = (address >> 8) & 255;  
    unsigned long d = address & 255;
```

```
    cout << "Extracted IPv4 address: "  
        << a << "." << b << "." << c << "." << d  
        << " (decimal value: " << address  
        << ", port: ";
```

```
    if (port == -1)  
        cout << "none";  
    else  
        cout << port;
```

```
    cout << ")\n";  
}
```

```
int main() {  
    string inputText;
```

```

cout << "Enter noisy string of text: ";
getline(cin, inputText);

unsigned long outAddress;
int outPort;

bool found = extractIPv4(inputText, outAddress, outPort);

if (found) {
    printIPv4(outAddress, outPort);
}
else {
    cout << "No address found.\n";
}

return 0;
}

```

This code does not include a loop to continuously prompt for input until “END” is entered.

Sept. 27

Prompt:

reformat this function to return a string instead of printing to the console: void printIPv4(unsigned long address, int port) {

```

    unsigned long a = (address >> 24) & 255;
    unsigned long b = (address >> 16) & 255;
    unsigned long c = (address >> 8) & 255;
    unsigned long d = address & 255;

```

```

    cout << "Extracted IPv4 address: "
         << a << "." << b << "." << c << "." << d
         << " (decimal value: " << address
         << ", port: ";

```

```

    if (port == -1)
        cout << "none";
    else
        cout << port;

```

```
    cout << ")\n";  
}
```

Chat GPT Response:

```
string printIPv4(unsigned long address, int port) {  
    unsigned long a = (address >> 24) & 255;  
    unsigned long b = (address >> 16) & 255;  
    unsigned long c = (address >> 8) & 255;  
    unsigned long d = address & 255;
```

```
    stringstream output;
```

```
    output << "Extracted IPv4 address: "  
        << a << "." << b << "." << c << "." << d  
        << " (decimal value: " << address  
        << ", port: ";
```

```
    if (port == -1)  
        output << "none";  
    else  
        output << port;
```

```
    output << ");
```

```
    return output.str();  
}
```

You'll need:

```
#include <sstream>  
#include <string>
```

Then you can use it like:

```
string result = printIPv4(address, port);  
cout << result << endl;
```

Or, if you're returning it from another function, you can directly do:

```
return printIPv4(address, port);
```